



SPRING-BOOT 3.5 → 4.0 · MINI GUIDE



Testing: 3.5 → 4.0

Seven test-suite breaks, one clean build away

One cut from the full Spring Break Cheat Sheets. Every card verified against Spring Boot 3.5.16 and 4.0.7. Real error output, real fixes, from the spring-break test suite. Steve Poole.

About this mini guide

This is one cut from **Spring Break Cheat Sheets**, the field guide to every breaking change between Spring Boot 3.5 and 4.0. Each card covers one change, verified by running real code on 3.5.16 and 4.0.7. One change, one card, one page: the error you'll see, the fix, and the traps.

HOW TO READ A CARD

TIER	How it fails. Tier 1 won't build, Tier 2 won't start, Tier 3 behaves differently. Tier 3 is the dangerous one: nothing goes red, your data just changes shape.
IMPACT	Effort per occurrence once found. S is find-and-replace, M needs an hour of care, L needs a plan.
OPENREWRITE	The badge means the Boot 4.0 recipe fixes this one for you. Run it before touching anything by hand.
WHAT YOU'LL SEE	The real output, so you can match it against your own build.
THE FIX	Before and after code, plus the steps when a rename isn't enough.
WATCH OUT	The traps that make this change look like a different bug.
VERIFY	The command that proves you're done.

THE SERIES

Spring Batch: 3.5 → 4.0	Eight changes between your jobs and 4.0
Hibernate & JPA: 3.5 → 4.0	Ten data-layer changes, mostly the quiet kind
Security & SSO: 3.5 → 4.0	Seven auth changes you cannot ignore
Data & Messaging: 3.5 → 4.0	Nine changes across your stores and streams
Testing: 3.5 → 4.0	Seven test-suite breaks, one clean build away THIS GUIDE
Observability & Ops: 3.5 → 4.0	Six changes your dashboards will notice first
Jackson & JSON: 3.5 → 4.0	Nine JSON changes everyone hits
Web & HTTP: 3.5 → 4.0	Eight changes at the HTTP layer
Core Platform & Build: 3.5 → 4.0	Twenty platform changes underneath everything
The Big Jobs: L-Impact Changes	The six changes that eat your sprint

The full guide collects every change in one book, with a complete planning matrix and the OpenRewrite robot list. github.com/spoole167/spring-break



About HeroDevs

HeroDevs keeps critical open source software secure after the community stops patching it. Never-Ending Support (NES) covers CVEs in Spring Boot 3.5, Java, and dozens of other end-of-life frameworks, so your team migrates on its own schedule instead of the vendor's deadline.

This book exists because migrations fail from surprise more often than difficulty. Every card is a fact, not a guess: verified by running real code on Spring Boot 3.5.16 and 4.0.7.

The complete book, every subsystem in one volume, is coming to LeanPub. Watch github.com/spoole167/spring-break for the link.

herodevs.com

The Planning Matrix

Every change in this book, in book order. Impact is the effort per occurrence once found (S small, M moderate, L large), not how many you have. Auto means the OpenRewrite Boot 4.0 recipe handles it.

7 CHANGES

4 × TIER 1 - WON'T BUILD

3 × TIER 2 - WON'T START

TIER	CHANGE	IMPACT	AUTO
1	@MockBean / @SpyBean Removed	M	Y
1	Test-Slice Annotation Starters Relocated	S	Y
1	Testcontainers 2.0 Package Relocation	S	Y
1	TestRestTemplate Package Relocated	S	•
2	MockitoTestExecutionListener Removed	S	•
2	Spring Boot Spock Integration Removed	L	•
2	@SpringBootTest No Longer Auto-Configures MockMvc	S	•

The Robot List vs The Human List

The left column is what the OpenRewrite Boot 4.0 recipe fixes while you get coffee. The right column needs a person who understands your code. This split is the honest size of the migration: the renames are free, the behaviour changes are the work.

THE ROBOT LIST

Run the recipe, review the diff, done.

@MockBean / @SpyBean Removed

Test-Slice Annotation Starters Relocated

Testcontainers 2.0 Package Relocation

THE HUMAN LIST

Read the card. Check your code. Decide.

TestRestTemplate Package Relocated

MockitoTestExecutionListener Removed

Spring Boot Spock Integration Removed

@SpringBootTest No Longer Auto-Configures MockMvc

TIER 1

Tier 1 - Won't Build

4 cards

1. @MockBean / @SpyBean Removed
2. Test-Slice Annotation Starters Relocated
3. Testcontainers 2.0 Package Relocation
4. TestRestTemplate Package Relocated

@MockBean / @SpyBean Removed

Tier 1 - Won't Build

Testing

Impact: M

🔄 OpenRewrite

WHAT CHANGED

The `@MockBean` and `@SpyBean` annotations from `org.springframework.boot.test.mock.mockito` were deprecated in Spring Boot 3.4 and removed in 4.0. They are replaced by `@MockitoBean` and `@MockitoSpyBean` from `org.springframework.test.context.bean.override.mockito`, which live in Spring Framework 7 itself rather than Spring Boot.

WHY

The new annotations live in Spring Framework's test context, so they work in plain Framework tests as well as Boot tests. Their cleaner bean override mechanism also avoids some of the context caching issues that plagued `@MockBean`.

SCOPE CHECK

Search for `@MockBean` and `@SpyBean` across `src/test`. Every test class using these annotations needs updating. That can be dozens or hundreds of test files. Count them before starting.

WHAT YOU'LL SEE

```
$ mvn clean test
[ERROR] COMPILATION ERROR :
[ERROR] OrderServiceTest.java:[3,48] package
    org.springframework.boot.test.mock.mockito does not exist
[ERROR] OrderServiceTest.java:[18,6] cannot find symbol
    symbol: class MockitoBean
[INFO] BUILD FAILURE
---
The build stops at test-compile. No tests run.
```

THE FIX

Import change

BEFORE

```
import org.springframework.boot.test.mock.mockito.MockBean;
```

AFTER

```
import org.springframework.test.context.bean.override.mockito.MockitoBean;
```

BEFORE

```
import org.springframework.boot.test.mock.mockito.SpyBean;
```


AFTER

```
import org.springframework.test.context.bean.override.mockito.MockitoSpyBean;
```

Annotation usage

BEFORE

```
@MockBean  
private OrderRepository orderRepository;  
  
@SpyBean  
private NotificationService notificationService;
```

AFTER

```
@MockitoBean  
private OrderRepository orderRepository;  
  
@MockitoSpyBean  
private NotificationService notificationService;
```

HOW TO FIX

Find-and-replace annotations.

Replace `@MockBean` with `@MockitoBean` and `@SpyBean` with `@MockitoSpyBean`. Update the imports from `org.springframework.boot.test.mock.mockito` to `org.springframework.test.context.bean.override.mockito`.

OpenRewrite recipe.

The Spring Boot 4.0 OpenRewrite migration recipe handles this rename automatically across the entire test codebase.

Check for `@MockBean` attributes.

If you used `@MockBean(name = "...")` or `@MockBean(classes = ...)`, verify the equivalent attributes exist on `@MockitoBean`. The attribute names may differ slightly.

WATCH OUT

If you upgrade without running `mvn clean`, stale test classes compiled against 3.5 can still load and run. Spring no longer recognises the old annotations, so the mock fields silently stay `null` and the first symptom is a `NullPointerException` mid-test, which looks like a runtime injection failure. Run a clean build and the real compile failure appears.

The new annotations are in the Spring Framework package, not Spring Boot. If you have test utility classes that reference the old annotation by fully qualified name (e.g., in custom annotations or reflection), those references need updating too.

The `@MockitoBean` reset behaviour between tests may differ from `@MockBean`. If you relied on mocks being reset between test methods, verify that behaviour is preserved.

IDE templates and test generators may still produce `@MockBean`. Update your IDE templates after migrating.

Test-Slice Annotation Starters Relocated

Tier 1 - Won't Build

Testing

Impact: S

🔄 OpenRewrite

Spring Boot 4.0 relocated the test-slice annotations. The old packages (e.g. `org.springframework.boot.test.autoconfigure.orm.jpa`) no longer exist, so imports of `@DataJpaTest`, `@WebMvcTest`, and friends fail to compile.

WHAT CHANGED

Spring Boot 4.0 reorganised the test auto-configuration packages. The old test-slice packages, such as `org.springframework.boot.test.autoconfigure.orm.jpa`, no longer exist, so existing imports of `@DataJpaTest`, `@WebMvcTest`, and the other slice annotations fail to compile. The annotations now live in the new per-technology test modules.

WHY

The test auto-configuration was restructured to support the new bean override mechanism and to align the package structure with Spring Framework 7's test context changes.

SCOPE CHECK

Search test sources for imports from

```
org.springframework.boot.test.autoconfigure : every one of them fails to compile on 4.0. Also search for custom annotations that use @TypeExcludeFilters , @OverrideAutoConfiguration , or @ImportAutoConfiguration .
```

WHAT YOU'LL SEE

```
$ mvn clean test
[ERROR] COMPILATION ERROR :
[ERROR] UserRepositoryTest.java:[5,50] package
org.springframework.boot.test.autoconfigure.orm.jpa does not exist
[ERROR] UserRepositoryTest.java:[12,2] cannot find symbol
symbol: class DataJpaTest
[INFO] BUILD FAILURE
---
The build stops at test-compile. No tests run.
```

THE FIX

Custom test-slice annotation

BEFORE

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@BootstrapWith(SpringBootTestContextBootstrapper.class)
@OverrideAutoConfiguration(enabled = false)
@TypeExcludeFilters(CustomSliceTypeExcludeFilter.class)
@AutoConfigureCache
@AutoConfigureCustomService
@ImportAutoConfiguration
public @interface CustomServiceTest {
}
```

AFTER

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@BootstrapWith(SpringBootTestContextBootstrapper.class)
@OverrideAutoConfiguration(enabled = false)
@TypeExcludeFilters(CustomSliceTypeExcludeFilter.class)
@AutoConfigureCache
@AutoConfigureCustomService
@ImportAutoConfiguration
public @interface CustomServiceTest {
    // Annotation structure unchanged, but verify your
    // TypeExcludeFilter extends the correct base class
    // and the imported auto-configurations still resolve
}
```

HOW TO FIX

Update the test-slice imports.

Every test importing a slice annotation from the old `org.springframework.boot.test.autoconfigure` packages needs the import updated to the annotation's new home in the per-technology test modules. The Spring Boot 4.0 OpenRewrite recipe handles this across the whole test codebase; your IDE can re-resolve individual imports.

Update custom test-slice annotations.

If you wrote custom test-slice annotations (like `@CustomServiceTest`), verify that the `TypeExcludeFilter`, bootstrapper, and auto-configuration imports still resolve. Update fully qualified class references to match the new package locations, then run the full suite to confirm each slice still loads a thin context.

WATCH OUT

Once the imports compile again, check what context each slice loads. A misconfigured slice can silently load a full application context instead of a thin one, making tests pass but run much slower and test less precisely.

Libraries that provide custom test slices (Spring Cloud, Spring Modulith) need to be updated too. Check their Spring Boot 4.0 compatibility versions.

FURTHER INFO

Part of Spring Boot 4.0's modularisation: test auto-configuration moved out of the monolithic `spring-boot-test-autoconfigure` module into per-technology test modules (`spring-boot-data-jpa-test`, `spring-boot-webmvc-test`, and so on), and the annotation packages moved with it. Registrations still living in `spring.factories` are a separate break: see `spring-factories-autoconfig-imports`.

[Spring-Break Demo](#) [Testing changes in Spring Boot 4.0](#) [Spring Boot 4.0 Migration Guide](#)

Testcontainers 2.0 Package Relocation

Tier 1 - Won't Build

Testing

Impact: S

🔄 OpenRewrite

WHAT CHANGED

Testcontainers 2.0 resolved JPMS split-package problems by moving specialised containers into module-specific packages.

`PostgreSQLContainer` moved from `org.testcontainers.containers` to

`org.testcontainers.postgresql`. Similar moves apply to all database and middleware modules (Kafka, Redis, RabbitMQ, etc.).

`GenericContainer` remains in `org.testcontainers.containers`.

The Maven group ID (`org.testcontainers`) is unchanged.

WHY

JPMS forbids multiple JARs contributing to the same package (split packages). In Testcontainers 1.x every module dumped its containers into `org.testcontainers.containers`, making modular JARs impossible.

The 2.0 sub-package split is a prerequisite for Java 17+ module support.

SCOPE CHECK

Search for

`org.testcontainers.containers` in Java/Kotlin test sources. Any import of a specialised container (PostgreSQL, MySQL, Kafka, Redis, etc.) from that package needs updating to its new module sub-package. `GenericContainer` does not need changing.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/test/java/com/example/IntegrationTest.java:[4,52]
error: cannot find symbol
  symbol:   class PostgreSQLContainer
  location: package org.testcontainers.containers
```

THE FIX

pom.xml — BOM: group ID stays `org.testcontainers`; remove explicit version if using Boot parent

BEFORE

```
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers-bom</artifactId>
  <version>1.19.8</version>
  <type>pom</type>
  <scope>import</scope>
</dependency>
```

AFTER

```
<!-- Managed by Spring Boot 4.0 parent POM – remove explicit BOM if using starter-parent -->
```

Java imports — specialised containers move to module sub-packages

BEFORE

```
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.containers.KafkaContainer;
```

AFTER

```
import org.testcontainers.postgresql.PostgreSQLContainer;
import org.testcontainers.kafka.KafkaContainer;
```

GenericContainer stays put — no change needed

BEFORE

```
import org.testcontainers.containers.GenericContainer;
```

AFTER

```
import org.testcontainers.containers.GenericContainer; // unchanged
```

HOW TO FIX

Update Java imports for specialised containers.

For each database or middleware container, change the import from

`org.testcontainers.containers.XxxContainer` to the module-specific sub-package, e.g. `org.testcontainers.postgresql.PostgreSQLContainer`, `org.testcontainers.kafka.KafkaContainer`, `org.testcontainers.redis.RedisContainer`. Check the [Testcontainers 2.0 migration guide](#) for the full mapping.

Remove explicit BOM version if using Spring Boot parent.

Spring Boot 4.0's parent POM manages the Testcontainers 2.0 version. If you previously imported the Testcontainers BOM explicitly with a 1.x version, remove the version override to avoid conflicts.

WATCH OUT

The Maven group ID is still `org.testcontainers`; do not change it to `com.testcontainers`. Only the Java package paths inside the JARs changed.

If you use Testcontainers in a shared test library, update that module first. A mix of 1.x and 2.x imports on the classpath will cause `ClassNotFoundException` at runtime.

Spring Boot's `@ServiceConnection` annotation resolves container types by class. If you reference the old

```
org.testcontainers.containers.PostgreSQLContainer
import in a @ServiceConnection field, updating the
import is sufficient; no annotation change needed.
```

FURTHER INFO

Driven by Testcontainers 2.0, the version Spring Boot 4.0 now manages.

spring-break module: `testcontainers-class-relocation` [Testcontainers 2.0 migration guide](#)

TestRestTemplate Package Relocated

Tier 1 - Won't Build

TestRestTemplate moved to org.springframework.boot.testclient. Every test that imports the old package fails to compile.

Testing

Impact: S

WHAT CHANGED

`TestRestTemplate` was removed from `org.springframework.boot.test.web.client` and relocated to `org.springframework.boot.testclient`.

WHY

Spring Boot 4.0 modularised its testing support. Separating the REST test clients into their own module shrinks the core test-autoconfigure footprint.

SCOPE CHECK

Search all test sources for `org.springframework.boot.test.web.client.TestRestTemplate` and for `@Autowired TestRestTemplate` or `@Autowired private TestRestTemplate`. Every hit needs the import updated.

WHAT YOU'LL SEE

```
$ mvn test-compile
[ERROR] /src/test/java/com/example/MyControllerTest.java:[3,56]
  error: package org.springframework.boot.test.web.client does not exist
[ERROR] /src/test/java/com/example/MyControllerTest.java:[15,5]
  error: cannot find symbol
    symbol: class TestRestTemplate
```

THE FIX

Test class import

BEFORE

```
import org.springframework.boot.test.web.client.TestRestTemplate;
```

AFTER

```
import org.springframework.boot.testclient.TestRestTemplate;
```

HOW TO FIX

Update the import.

Replace `org.springframework.boot.test.web.client.TestRestTemplate` with `org.springframework.boot.testclient.TestRestTemplate`. The class itself is functionally equivalent.

Consider RestTestClient.

The new `RestTestClient` provides a more modern, fluent API for REST testing and is the preferred approach in Boot 4.0. If you are refactoring tests anyway, this is a good time to migrate.

WATCH OUT

Custom configuration classes that create a `TestRestTemplate` bean (for authentication headers, say) need the import updated too, alongside the test classes that inject it.

FURTHER INFO

Part of Spring Boot 4.0's testing overhaul: the class now ships in the `spring-boot-test-rest-client` module rather than `spring-boot-test-autoconfigure`. See also: `springboottest-no-mockmvc`.

spring-break module: `testrest-template-removed` [Spring Boot 4.0 Migration Guide](#)

TIER 2

Tier 2 - Won't Start

3 cards

1. MockitoTestExecutionListener Removed
2. Spring Boot Spock Integration Removed

3. @SpringBootTest No Longer Auto-Configures MockMvc

MockitoTestExecutionListener Removed

Tier 2 - Won't Start

Spring Boot 4.0 removes MockitoTestExecutionListener. @Mock fields in @SpringBootTest tests stay null and throw NullPointerException unless @ExtendWith(MockitoExtension.class) is added.

Testing

Impact: S

WHAT CHANGED

MockitoTestExecutionListener was a Spring Boot-specific bridge that triggered Mockito's annotation processing inside Spring's test lifecycle. Spring Boot 4.0 removed this listener, aligning with the standard JUnit 5 approach of using @ExtendWith(MockitoExtension.class) or MockitoAnnotations.openMocks(this) in a @BeforeEach setup method.

WHY

The listener duplicated Mockito's own JUnit 5 extension. Removing it pushes tests toward the idiomatic Mockito + JUnit 5 pattern that works in any JUnit 5 project.

SCOPE CHECK

Search for @Mock, @Spy, and @Captor annotations in test classes that also use @SpringBootTest or any Spring test slice annotation (@WebMvcTest, @DataJpaTest, etc.). Each such class needs @ExtendWith(MockitoExtension.class) added.

WHAT YOU'LL SEE

```
@SpringBootTest
class MyTest {
    @Mock
    private MyService myService;

    @Test
    void test() {
        // Boot 3.5: myService is initialised → passes
        // Boot 4.0: myService is null → NullPointerException
        when(myService.greet()).thenReturn("Hello");
    }
}

java.lang.NullPointerException: Cannot invoke method on null object
at com.example.MyTest.test(MyTest.java:15)
```

THE FIX

Add MockitoExtension alongside @SpringBootTest

BEFORE

```
@SpringBootTest
class MyTest {
    @Mock
    private MyService myService;
}
```

AFTER

```
@SpringBootTest
@ExtendWith(MockitoExtension.class)
class MyTest {
    @Mock
    private MyService myService;
}
```

Or switch to `@MockitoBean` (Boot 4.0 replacement for `@MockBean`)

BEFORE

```
@SpringBootTest
class MyTest {
    @Mock
    private MyService myService;
}
```

AFTER

```
@SpringBootTest
class MyTest {
    @MockitoBean
    private MyService myService; // registered in the Spring context
}
```

HOW TO FIX

Add `@ExtendWith(MockitoExtension.class)` to the test class.

This is the standard JUnit 5 way to enable Mockito annotation processing. It works alongside `@SpringBootTest` and needs no new dependency: Mockito's JUnit 5 extension is already on the test classpath via `spring-boot-starter-test`.

Use `@MockitoBean` instead of `@Mock` (if the mock belongs in the context).

If the mock is being injected into a Spring bean under test, use `@MockitoBean` (Boot 4.0's replacement for `@MockBean`). This registers the mock in the Spring application context rather than just the test instance.

VERIFY

`@Mock` fields in `@SpringBootTest` classes are initialised correctly

WATCH OUT

If a test class extends another that has `@Mock` fields, the parent class also needs the extension. Inheritance doesn't propagate the `@ExtendWith` from child to parent field initialisation.

Tests that used `@MockBean` (Boot 3.5) are affected by a separate change: `@MockBean` itself is removed in Boot 4.0 in favour of `@MockitoBean`. See the [mockbean-removed](#) card.

FURTHER INFO

The listener called `MockitoAnnotations.openMocks()` on each test instance, initialising `@Mock`, `@Spy`, and `@Captor` fields before each test. Mockito's own JUnit 5 extension has done the same job since Mockito 3.x.

Spring Boot Spock Integration Removed

Tier 2 - Won't Start

Spring Boot 4.0 requires Groovy 5; Spock does not support it yet. Spock-based

`@SpringBootTest` specifications stop running.

Testing

Impact: L

WHAT CHANGED

Spring Boot 4.0 upgraded its Groovy baseline to Groovy 5. Spock Framework (as of this writing) requires Groovy 4.x or earlier, so Spring Boot removed the Spock integration.

WHY

The Groovy 5 upgrade aligned Boot with the current Groovy ecosystem. Spock's Groovy 5 support was not ready in time for the Boot 4.0 release, making the integration impossible to ship.

SCOPE CHECK

Find all `.groovy` test files that extend `Specification` and carry `@SpringBootTest`, `@SpringRunner`, or any Spring test slice annotation. These are the tests that break.

WHAT YOU'LL SEE

```
// Spock specification in a Boot 4.0 project:
@SpringBootTest
class MyServiceSpec extends Specification {
    @Autowired MyService service

    def "should do something"() {
        expect: service.doThing() == "result"
    }
}

// Fails at compile or runtime — Spock and Groovy 5 are incompatible.
// Exact error depends on Groovy/Spock versions on the classpath.
```

THE FIX

Spock specification → JUnit 5 migration

BEFORE

```
// MyServiceSpec.groovy
@SpringBootTest
class MyServiceSpec extends Specification {
    @Autowired MyService service

    def "should return result"() {
        expect: service.doThing() == "result"
    }
}
```

AFTER

```
// MyServiceTest.java
@SpringBootTest
class MyServiceTest {
    @Autowired MyService service;

    @Test
    void shouldReturnResult() {
        assertThat(service.doThing()).isEqualTo("result");
    }
}
```

HOW TO FIX

Migrate Spock specifications to JUnit 5.

Rewrite Spock-based Spring integration tests using JUnit 5 and AssertJ. The `given/when/then` blocks map to JUnit 5 methods and AssertJ assertions. Data-driven tests (`where:` blocks) translate to `@ParameterizedTest` with `@MethodSource` or `@CsvSource`.

Wait for Spock's Groovy 5 support if migration is impractical.

If the volume of Spock tests makes immediate migration impractical, track the Spock project's Groovy 5 roadmap and defer the Boot 4.0 upgrade for those modules.

VERIFY

Spock specifications that use `@SpringBootTest` or `@SpringRunner` pass after migrating to JUnit 5 or waiting for Spock's Groovy 5 support

WATCH OUT

Pure unit Spock tests, those that never load a Spring context, still work as long as Groovy compilation itself succeeds. The breakage is in tests that depend on Spring Boot's test infrastructure.

FURTHER INFO

The removal covers the `spock-spring` module and Boot's `@SpringBootTest` support for Spock specifications. Failures appear at Groovy compile time or at runtime before any test method executes.

@SpringBootTest No Longer Auto-Configures MockMvc

Tier 2 - Won't Start

Testing

Impact: S

`@SpringBootTest` no longer auto-configures `MockMvc` in Boot 4.0. Add `@AutoConfigureMockMvc` to your test class or the `MockMvc` bean won't be available.

WHAT CHANGED

`MockMvc` auto-configuration was decoupled from `@SpringBootTest`. The `Mock` webEnvironment still creates a mock servlet context, but the `MockMvc` bean must now be opted into with `@AutoConfigureMockMvc`.

WHY

The implicit bean was a hidden dependency: tests got `MockMvc` without declaring they needed it. Opt-in matches Boot 4.0's explicit-over-implicit theme and keeps the test context leaner when `MockMvc` isn't needed.

SCOPE CHECK

Search for `@SpringBootTest` test classes that also autowire `MockMvc` (look for `@Autowired` fields or constructor parameters of type `MockMvc`). Each needs `@AutoConfigureMockMvc` added. `@WebMvcTest` tests are unaffected.

WHAT YOU'LL SEE

```
@SpringBootTest
class MyControllerTest {
    @Autowired
    MockMvc mockMvc; // null in Boot 4.0

    @Test
    void test() throws Exception {
        mockMvc.perform(get("/api/hello")) // NullPointerException
            .andExpect(status().isOk());
    }
}

// Boot 4.0 failure:
org.springframework.beans.factory.NoSuchBeanDefinitionException:
No qualifying bean of type
'org.springframework.test.web.servlet.MockMvc' available
```

THE FIX

Add `@AutoConfigureMockMvc`

BEFORE

```
@SpringBootTest
class MyControllerTest {
    @Autowired
    MockMvc mockMvc;
}
```

AFTER

```
@SpringBootTest
@AutoConfigureMockMvc
class MyControllerTest {
    @Autowired
    MockMvc mockMvc;
}
```

@WebMvcTest is unaffected — MockMvc is its primary purpose

ADDED

```
// @WebMvcTest already includes MockMvc auto-configuration.
// Only @SpringBootTest is affected.
```

HOW TO FIX

Add @AutoConfigureMockMvc to the test class.

This is a one-annotation fix. Add `@AutoConfigureMockMvc` alongside `@SpringBootTest` on any test class that autowires `MockMvc`.

VERIFY

MockMvc is available in @SpringBootTest tests after adding @AutoConfigureMockMvc

WATCH OUT

The same change applies to `WebTestClient` and `TestRestTemplate`. Boot 4.0 also no longer auto-configures these in `@SpringBootTest`: add `@AutoConfigureRestTestClient` or `@AutoConfigureTestRestTemplate` respectively.

FURTHER INFO

Per the migration guide, a Boot 3.5 `@SpringBootTest` with the default `MockWebEnvironment` supplied `MockMvc` without any extra annotation; tests relying on that fail with `NoSuchBeanDefinitionException` on 4.0.

spring-break module: springboottest-mockmvc-autoconfig Spring Boot 4.0 Migration Guide



Not ready to migrate?

Spring Boot 3.5 stops receiving fixes when open-source support ends. You have three sane options. Pick one deliberately.

1. **Pin 3.5 and track CVEs.** Free, but the clock is running and every new CVE is your team's problem to triage.
2. **Migrate incrementally with OpenRewrite.** The Boot 4.0 recipe automates the renames in this book; the behavioural changes still need eyes.
3. **HeroDevs NES for Spring.** Security patches for 3.5 beyond EOL, buying your team time to migrate on your schedule, not the calendar's.

Spring Boot 4.0 Migration Guide • github.com/spring-projects/spring-boot/wiki
OpenRewrite Boot 4.0 recipe • docs.openrewrite.org/recipes/java/spring/boot4
HeroDevs NES for Spring • herodevs.com/support/spring-nes
The test suite behind this book • github.com/spoole167/spring-break